In [1]:
```
import pandas as pd
import seaborn as sns
```

In [7]:
```
data = pd.read_csv("breast_cancer.csv")
data.head(10)
```

Out[7]:

| | id | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | smoothn |
|---|---|---|---|---|---|---|---|
| **0** | 842302 | M | 17.99 | 10.38 | 122.80 | 1001.0 | |
| **1** | 842517 | M | 20.57 | 17.77 | 132.90 | 1326.0 | |
| **2** | 84300903 | M | 19.69 | 21.25 | 130.00 | 1203.0 | |
| **3** | 84348301 | M | 11.42 | 20.38 | 77.58 | 386.1 | |
| **4** | 84358402 | M | 20.29 | 14.34 | 135.10 | 1297.0 | |
| **5** | 843786 | M | 12.45 | 15.70 | 82.57 | 477.1 | |
| **6** | 844359 | M | 18.25 | 19.98 | 119.60 | 1040.0 | |
| **7** | 84458202 | M | 13.71 | 20.83 | 90.20 | 577.9 | |
| **8** | 844981 | M | 13.00 | 21.82 | 87.50 | 519.8 | |
| **9** | 84501001 | M | 12.46 | 24.04 | 83.97 | 475.9 | |

10 rows × 33 columns

In [8]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

In [9]:
```
data.describe()
```

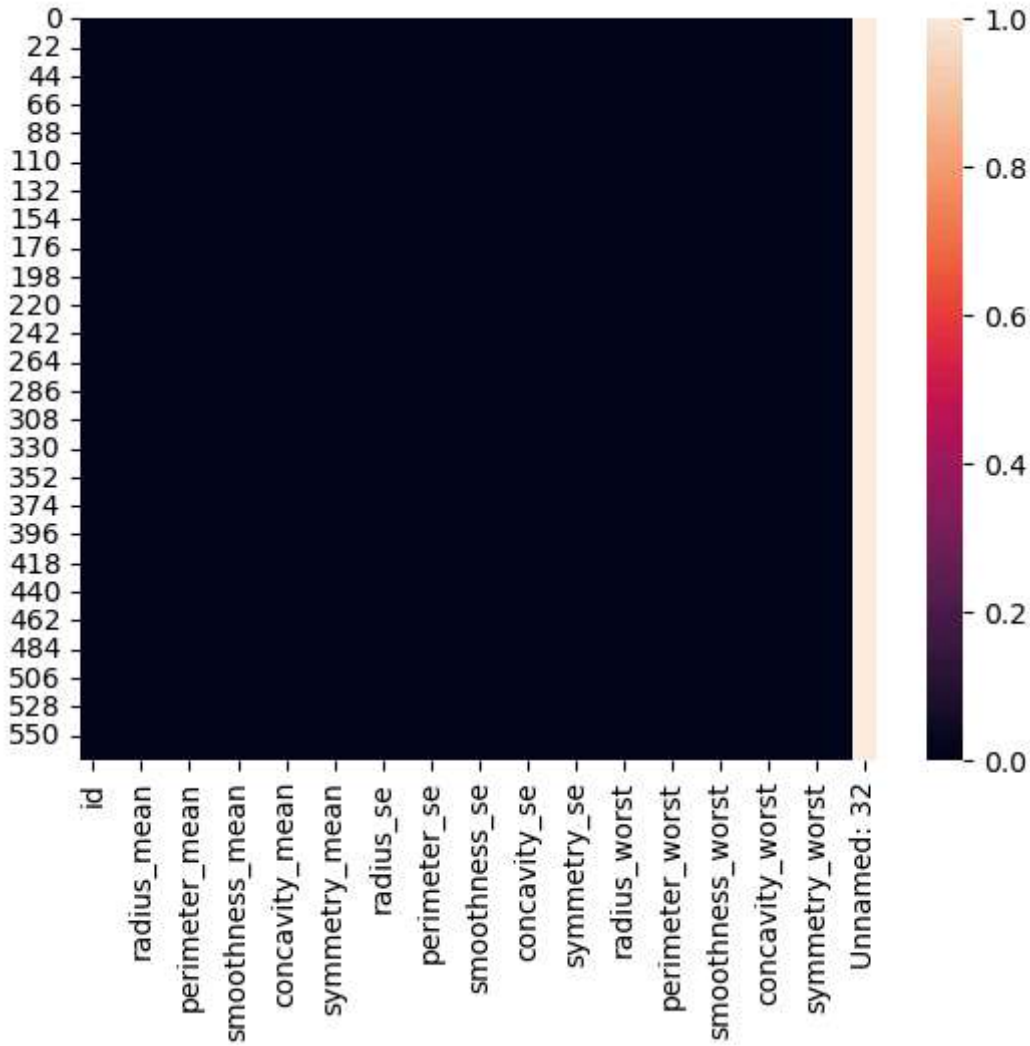
Out[9]:

| | id | radius_mean | texture_mean | perimeter_mean | area_mean | smoothnes |
|---|---|---|---|---|---|---|
| **count** | 5.690000e+02 | 569.000000 | 569.000000 | 569.000000 | 569.000000 | 569 |
| **mean** | 3.037183e+07 | 14.127292 | 19.289649 | 91.969033 | 654.889104 | 0 |
| **std** | 1.250206e+08 | 3.524049 | 4.301036 | 24.298981 | 351.914129 | 0 |
| **min** | 8.670000e+03 | 6.981000 | 9.710000 | 43.790000 | 143.500000 | 0 |
| **25%** | 8.692180e+05 | 11.700000 | 16.170000 | 75.170000 | 420.300000 | 0 |
| **50%** | 9.060240e+05 | 13.370000 | 18.840000 | 86.240000 | 551.100000 | 0 |
| **75%** | 8.813129e+06 | 15.780000 | 21.800000 | 104.100000 | 782.700000 | 0 |
| **max** | 9.113205e+08 | 28.110000 | 39.280000 | 188.500000 | 2501.000000 | 0 |

8 rows × 32 columns

In [21]:
```
sns.heatmap(data.isnull())
```

Out[21]:
```
<Axes: >
```




In [23]:
```
data.drop(["Unnamed: 32", "id"], axis=1, inplace=True)
data.head()
```

Out[23]:

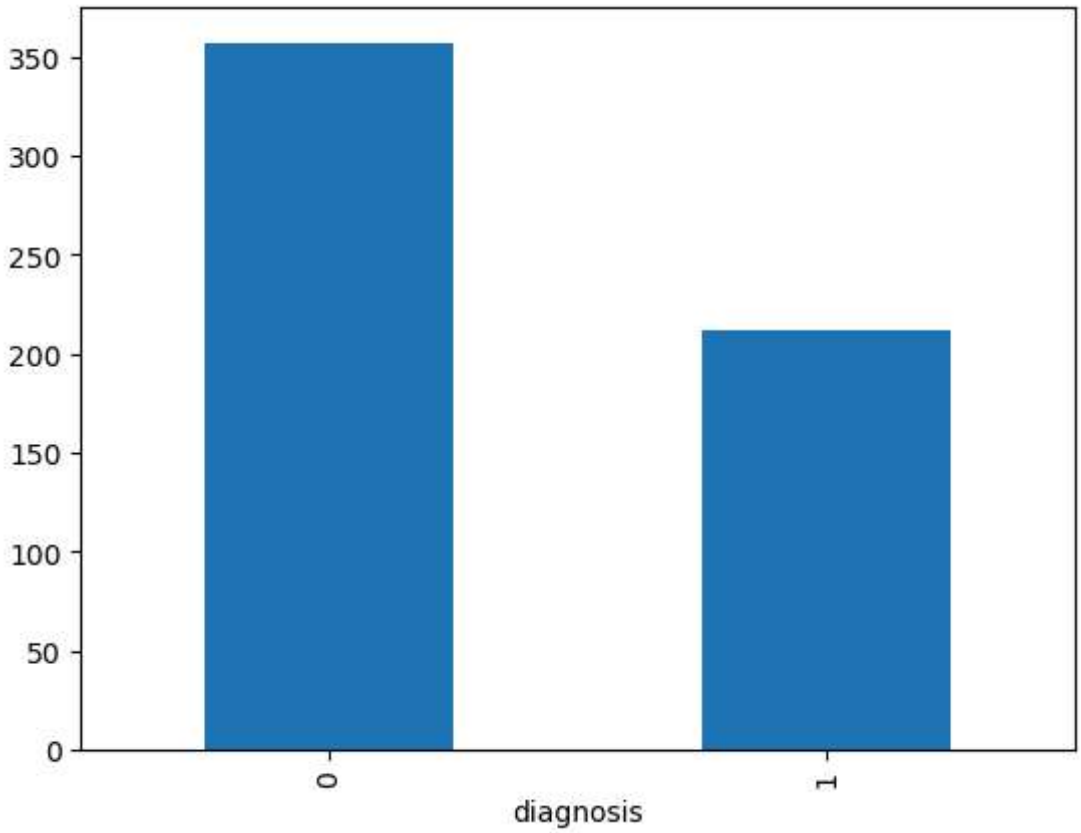
| | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | smoothness_mean |
|---|---|---|---|---|---|---|
| **0** | M | 17.99 | 10.38 | 122.80 | 1001.0 | 0.11840 |
| **1** | M | 20.57 | 17.77 | 132.90 | 1326.0 | 0.08474 |
| **2** | M | 19.69 | 21.25 | 130.00 | 1203.0 | 0.10960 |
| **3** | M | 11.42 | 20.38 | 77.58 | 386.1 | 0.14250 |
| **4** | M | 20.29 | 14.34 | 135.10 | 1297.0 | 0.10030 |

5 rows × 31 columns

In [31]:
```
data.diagnosis = [1 if value =="M" else 0 for value in data.diagnosis]
```

In [45]:
```
data['diagnosis'] = data['diagnosis'].astype("category", copy=False)
data["diagnosis"].value_counts().plot(kind="bar")
```

Out[45]: <Axes: xlabel='diagnosis'>




In [64]:
```
#divide into target variable and predictors
y = data["diagnosis"] # our target variable
x = data.drop(["diagnosis"], axis=1)
```

In [65]:
```
x
```

Out[65]:

| | radius_mean | texture_mean | perimeter_mean | area_mean | smoothness_mean | compacti |
|---|---|---|---|---|---|---|
| **0** | 17.99 | 10.38 | 122.80 | 1001.0 | 0.11840 | |
| **1** | 20.57 | 17.77 | 132.90 | 1326.0 | 0.08474 | |
| **2** | 19.69 | 21.25 | 130.00 | 1203.0 | 0.10960 | |
| **3** | 11.42 | 20.38 | 77.58 | 386.1 | 0.14250 | |
| **4** | 20.29 | 14.34 | 135.10 | 1297.0 | 0.10030 | |
| **...** | ... | ... | ... | ... | ... | |
| **564** | 21.56 | 22.39 | 142.00 | 1479.0 | 0.11100 | |
| **565** | 20.13 | 28.25 | 131.20 | 1261.0 | 0.09780 | |
| **566** | 16.60 | 28.08 | 108.30 | 858.1 | 0.08455 | |
| **567** | 20.60 | 29.33 | 140.10 | 1265.0 | 0.11780 | |
| **568** | 7.76 | 24.54 | 47.92 | 181.0 | 0.05263 | |

569 rows × 30 columns

# normalize the data

In [66]:
```
from sklearn.preprocessing import StandardScaler

# create scaler object
scaler = StandardScaler()

# fit and transform data
x_scaled = scaler.fit_transform(x)
```

In [67]:
```
x_scaled
```

Out[67]:
```
array([[ 1.09706398, -2.07333501,  1.26993369, ...,  2.29607613,
         2.75062224,  1.93701461],
       [ 1.82982061, -0.35363241,  1.68595471, ...,  1.0870843 ,
        -0.24388967,  0.28118999],
       [ 1.57988811,  0.45618695,  1.56650313, ...,  1.95500035,
         1.152255  ,  0.20139121],
       ...,
       [ 0.70228425,  2.0455738 ,  0.67267578, ...,  0.41406869,
        -1.10454895, -0.31840916],
       [ 1.83834103,  2.33645719,  1.98252415, ...,  2.28998549,
         1.91908301,  2.21963528],
       [-1.80840125,  1.22179204, -1.81438851, ..., -1.74506282,
        -0.04813821, -0.75120669]], shape=(569, 30))
```

# Split the data

In [68]:
```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.30, ra
```

# train the model

In [69]:
```
from sklearn.linear_model import LogisticRegression

#create the lr model
lr = LogisticRegression()

#train the model on the training data
lr.fit(x_train, y_train)

#predict the target variable on test data
y_pred = lr.predict(x_test)
```

# evaluation of the model

In [72]:
```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy: .2f}")
```

```
Accuracy:  0.98
```

In [73]:
```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.99      0.98      0.99       108
           1       0.97      0.98      0.98        63

    accuracy                           0.98       171
   macro avg       0.98      0.98      0.98       171
weighted avg       0.98      0.98      0.98       171
```

In [74]:
```
from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_test, y_pred))
```

```
[[106   2]
 [  1  62]]
```

In [76]:
```
import pandas as pd

coefficients = pd.DataFrame({
    "Feature": x.columns,
    "Coefficient": lr.coef_[0]
```

```
})

print(coefficients)
```

```
                   Feature  Coefficient
0               radius_mean     0.367670
1              texture_mean     0.368886
2            perimeter_mean     0.323265
3                 area_mean     0.424750
4           smoothness_mean     0.184647
5          compactness_mean    -0.595640
6            concavity_mean     0.769889
7       concave points_mean     1.094419
8             symmetry_mean    -0.240961
9    fractal_dimension_mean    -0.142817
10                radius_se     1.207227
11               texture_se    -0.169539
12             perimeter_se     0.565605
13                  area_se     0.881542
14            smoothness_se     0.171874
15           compactness_se    -0.619085
16             concavity_se     0.107181
17        concave points_se     0.464623
18              symmetry_se    -0.514638
19     fractal_dimension_se    -0.695701
20             radius_worst     0.806112
21            texture_worst     1.323063
22          perimeter_worst     0.533924
23               area_worst     0.772227
24         smoothness_worst     0.508286
25        compactness_worst    -0.100956
26          concavity_worst     0.974141
27     concave points_worst     0.835418
28           symmetry_worst     1.205759
29  fractal_dimension_worst     0.115292
```

In [77]:

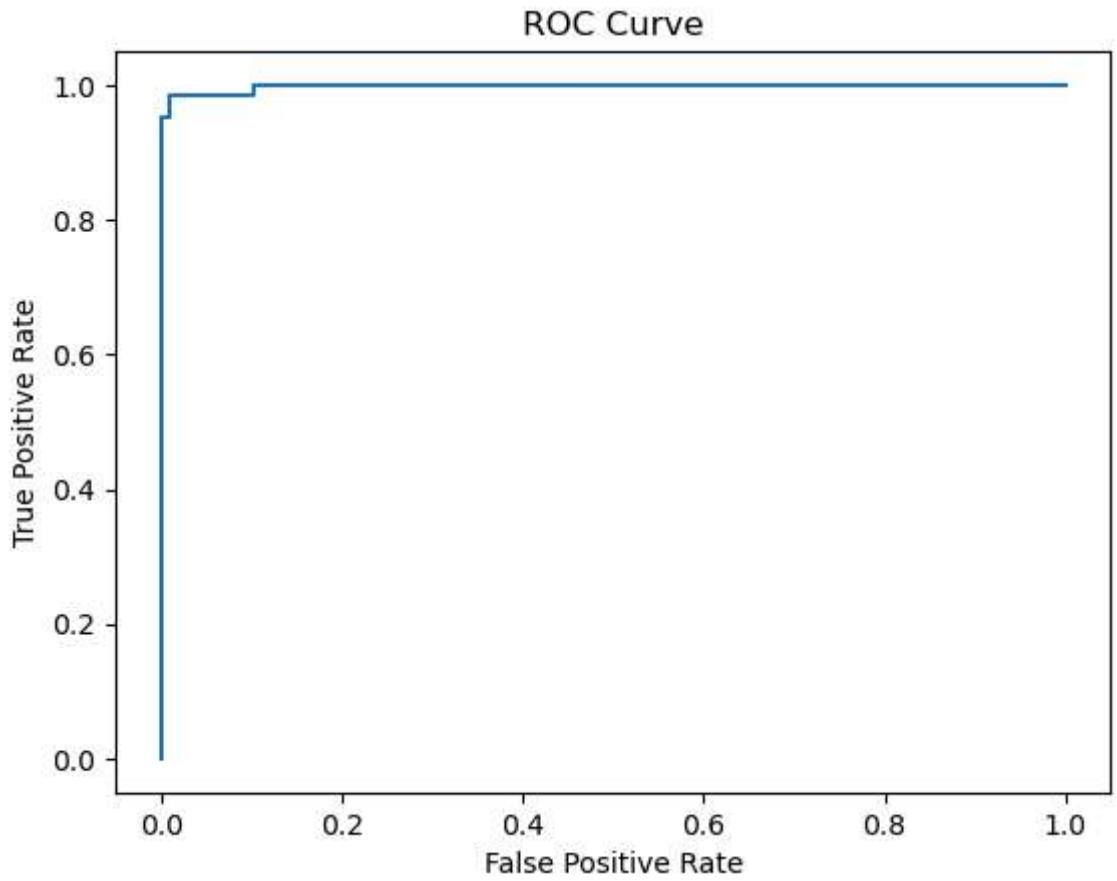
```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

y_prob = lr.predict_proba(x_test)[:,1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()

print("AUC Score:", roc_auc_score(y_test, y_prob))
```




```
AUC Score: 0.9980893592004703
```

In [78]:
```
import joblib

joblib.dump(lr, "breast_cancer_model.pkl")
```

Out[78]:
```
['breast_cancer_model.pkl']
```